

Delegates und Events

Delegates

- Delegate-Variablen speichern Referenzen auf andere Methoden
- die Referenzen können zur Laufzeit hinzugefügt oder entfernt werden
- die referenzierten Methoden müssen die gleiche Signatur (gleicher Aufbau) wie der Delegate-Typ haben

```
public delegate string MeinDelegateTyp(string param1, int param2);

public string Vorstellung(string name, int alter)
{
    return $"Hallo, ich heiße {name} und bin {alter} Jahre alt.";
}

MeinDelegateTyp meinDelegate = new MeinDelegateTyp(Vorstellung);
meinDelegate("Max Mustermann", 37);
//Hallo, ich heiße Max Mustermann und bin 37 Jahre alt.
```

Delegates

- Jede Delegate-Variable kann beliebig viele Methoden referenzieren, welche nacheinander ausgeführt werden
- Methoden können durch += hinzugefügt und durch -= herausgelöst werden

```
public delegate void MeinDelegateTyp(string param1, int param2);

public void VorstellungDeu(string name, int alter)
{
    Console.WriteLine($"Hallo, ich heiße {name} und bin {alter} Jahre alt.");
}
public void VorstellungEng(string name, int age)
{
    Console.WriteLine($"Hello, my name is {name} and I am {age} years old.");
}
public void VorstellungFra(string nom, int age)
{
    Console.WriteLine($"Bonjour, je m'appelle {nom} et j'ai {age} ans.");
}
public void VorstellungSpa(string nombre, int anos)
{
    Console.WriteLine($"Hola, mi nombre es {nombre} y tengo {anos} años.");
}
```

```
MeinDelegateTyp meinDelegate = new MeinDelegateTyp(VorstellungDeu);
meinDelegate += new MeinDelegateTyp(VorstellungEng);
meinDelegate += new MeinDelegateTyp(VorstellungFra);
meinDelegate += new MeinDelegateTyp(VorstellungSpa);

meinDelegate("Max Mustermann", 37);
//Hallo, ich heiße Max Mustermann und bin 37 Jahre alt.
//Hello, my name is Max Mustermann and I am 37 years old.
//Bonjour, je m'appelle Max Mustermann et j'ai 37 ans.
//Hola, mi nombre es Max Mustermann y tengo 37 años.
```

vordefinierte Delegate-Typen

Name	Rückgabetyp	Parameter	Beispiel für referenzierbare Methode
Action Action<in T1> Action<in T1, in T2> ...	void	0-16	<pre>public void PrintInteger(int p1) { Console.Write(p1); }</pre> Action<int> a = PrintInteger
Predicate<in T>	bool	1	<pre>public bool CheckForZero(int p1) { return p1 == 0; }</pre> Predicate<int> p = CheckForZero
Func<out TResult> Func<in T1, out TResult> Func<in T1, in T2, out TResult> ...	TResult	0-16	<pre>public string SumToString(int p1, int p2) { return (p1 + p2).ToString(); }</pre> Func<int, int, string> f = SumToString

anonyme Methoden

- Methoden ohne Namen nennt man anonyme Methoden
- Methoden können einem Delegate zugewiesen werden, ohne diese vorher zu deklarieren

```
Func<int, int> meineFunc = delegate (int param1)
{
    return param1 + 5;
};
```

- ‚Wegwerfmethoden‘ für einmaligen Gebrauch, z.B. zur Übergabe zusätzlicher Logik an Methoden

```
List<string> StädteListe = new List<string>() { "München", "Berlin", "Hamburg", "Hannover", "Köln" };
string gefundeneStadt = StädteListe.Find
(
    delegate (string stadt)
    {
        return stadt.StartsWith('B');
    }
);

// => gefundeneStadt = "Berlin"
```

Lambda - Ausdrücke

- Kurzschreibweise für anonyme Methoden

```
Func<int, int, int> Addiere = delegate(int x, int y) {return x+y;}
```



```
Func<int, int , int> Addiere = (int x, int y) => {return x+y;}
```



```
Func<int, int , int> Addiere = (x, y) => x+y;
```

EventHandler

- ein Event ist eine Delegate-Variable, an der sich beliebig viele Methoden (EventListener) an- und abmelden können
- Delegate-Typ ist i.d.R. EventHandler
 - Erwartete Übergabeparameter: `object`, `EventArgs`
- Kein ‚Überschreiben‘ durch Neuzuweisung möglich

```
event EventHandler onClick;
```

```
onClick += delegate (object sender, EventArgs EventArgs)  
{  
    Console.WriteLine("EventBehandlung");  
};
```

```
onClick(this, EventArgs.Empty);
```